

Практическое задание 4

Работа с устройствами

<https://github.com/ananass12/Real-time-systems>

Задание 1

Задание 1: Чтение событий с устройства ввода

Напишите программу `read_input`, которая открывает одно из устройств ввода (`/dev/input/eventX`) и в бесконечном цикле читает из него структуры `struct input_event`. Программа должна выводить на экран поля каждой структуры: `type`, `code`, `value`.

Требования:

1. Путь к файлу устройства должен передаваться через аргумент командной строки.
2. Программа должна проверять, что у пользователя есть права на чтение из указанного файла.
3. Для корректной работы вам понадобится заголовочный файл `<linux/input.h>`.
4. Как найти нужное устройство? Выполните команду `cat /proc/bus/input/devices` и найдите секцию, соответствующую вашей клавиатуре. В строке `Handlers=` будет указан `eventX`, который вам нужен.

`read_input`:

1. Принимает путь к устройству (например, `/dev/input/event3`) как аргумент.
2. Открывает его с помощью `open()`.
3. В бесконечном цикле читает структуры `struct input_event` через `read()`.
4. Выводит поля: `type`, `code`, `value`.

- `type` - тип события
- `code` - код ключа, оси или прочего
- `value` - значение (нажато/отпущено, смещение и т.д.)

Тип (type)	Константа	Значение
0	EV_SYN	Синхронизация (завершает пакет событий)
1	EV_KEY	Нажатие/отпускание клавиши или кнопки
4	EV_MSC	Прочая информация (например, scan-код)

KEY_SPACE	57
KEY_ENTER	28
KEY_PAUSE	99

```
gcc -Wall -Wextra -std=c99 -o read_input src/read_input.c
root@Ubuntu:/home/nastya/RTS/tasks/task4# ./read_input /dev/input/event3
Reading events from /dev/input/event3. Press Ctrl+C to exit.
Event: type 4, code 4, value 28
Event: type 1, code 28, value 0
Event: type 0, code 0, value 0
Event: type 4, code 4, value 32
Event: type 1, code 32, value 1
Event: type 0, code 0, value 0
Event: type 4, code 4, value 32
Event: type 1, code 32, value 0
Event: type 0, code 0, value 0
Event: type 4, code 4, value 32
Event: type 1, code 32, value 1
Event: type 0, code 0, value 0
Event: type 4, code 4, value 32
Event: type 1, code 32, value 0
Event: type 0, code 0, value 0
```

Задание 2

Задание 2: Использование `ioctl` для получения информации об устройстве

Модифицируйте программу `read_input` из предыдущего задания. Перед началом чтения событий, используйте системный вызов `ioctl` для получения и вывода в консоль имени устройства.

Требования:

1. Используйте команду `EVIOCGNAME(len)` для получения имени устройства. `len` — это размер буфера для имени.
2. Программа должна корректно обрабатывать возможные ошибки вызова `ioctl`.
3. В дополнение к имени, попробуйте получить и вывести физический путь устройства с помощью `EVIOCGPHYS(len)`.

Добавляем:

- Вызов `ioctl(fd, EVIOCGNAME, ...)` - получаете читаемое имя устройства («AT Translated Set 2 keyboard»).
- Вызов `ioctl(fd, EVIOCGPHYS, ...)` - получаете физический путь в шине («isa0060/serio0/input0»).

Это нужно чтобы понять, с каким именно устройством вы работаете, особенно если их много.

```

// Вывести имя устройства
char name[256] = "Unknown";
if (ioctl(fd, EVIOCGNAME(sizeof(name)), name) < 0) {
    perror("Warning: ioctl(EVIOCGNAME) failed");
    // Продолжаем работу, имя останется "Unknown"
} else {
    printf("Device name: %s\n", name);
}

//Дополнительно физический путь
char phys[256] = "Unknown";
if (ioctl(fd, EVIOCGPHYS(sizeof(phys)), phys) < 0) {
    // Не критично, многие устройства не выставляют phys
} else {
    printf("Physical path: %s\n", phys);
}

```

```

Usage: ./read_input /dev/input/eventX
root@Ubuntu:/home/nastya/RTS/tasks/task4# ./read_input /dev/input/event3
Device name: AT Translated Set 2 keyboard
Physical path: isa0060/serio0/input0
Reading events from /dev/input/event3. Press Ctrl+C to exit.
Event: type 4, code 4, value 28
Event: type 1, code 28, value 0
Event: type 0, code 0, value 0
Event: type 4, code 4, value 183
Event: type 1, code 99, value 1
Event: type 0, code 0, value 0
Event: type 4, code 4, value 183
Event: type 1, code 99, value 0
Event: type 0, code 0, value 0
Event: type 4, code 4, value 29
Event: type 1, code 29, value 1
Event: type 0, code 0, value 0
Event: type 4, code 4, value 47
Event: type 1, code 47, value 1
Event: type 0, code 0, value 0
Event: type 4, code 4, value 29
Event: type 1, code 29, value 0
Event: type 0, code 0, value 0
Event: type 4, code 4, value 47
Event: type 1, code 47, value 0
Event: type 0, code 0, value 0

```


Задание 3

Задание 3: Мониторинг нескольких устройств с помощью poll

Напишите новую программу `poll_inputs`, которая может одновременно отслеживать события от нескольких устройств ввода.

Требования:

1. Программа должна принимать несколько путей к устройствам в качестве аргументов командной строки (например, `./poll_inputs /dev/input/event2 /dev/input/event4`).
2. Для каждого переданного устройства откройте файловый дескриптор.
3. Используйте системный вызов `poll()` для ожидания событий на любом из открытых дескрипторов.
4. Когда `poll()` сообщает о событии, определите, на каком именно дескрипторе оно произошло, прочитайте `struct input_event` и выведите информацию о событии, а также имя устройства, с которого оно пришло (используйте `ioctl` из Задания 2).

`poll_inputs`:

- Принимает несколько путей к устройствам.
- Открывает все с флагом `O_NONBLOCK` (чтобы `read()` не блокировал).
- Использует `poll()` для ожидания данных на любом из дескрипторов.
- Как только приходит событие — определяет, с какого устройства, и выводит его с именем.

`poll()` — эффективный, масштабируемый, энергосберегающий способ ожидания событий от множества источников.

```
root@Ubuntu:/home/nastya/RTS/tasks/task4# ./poll_inputs /dev/input/event3 /dev/input/event4
Polling 2 devices. Press Ctrl+C to exit.
[AT Translated Set 2 keyboard] type=4, code=4, value=28
[AT Translated Set 2 keyboard] type=1, code=28, value=0
[AT Translated Set 2 keyboard] type=0, code=0, value=0
[AT Translated Set 2 keyboard] type=4, code=4, value=200
[AT Translated Set 2 keyboard] type=1, code=103, value=1
[AT Translated Set 2 keyboard] type=0, code=0, value=0
^[[A[AT Translated Set 2 keyboard] type=4, code=4, value=200
[AT Translated Set 2 keyboard] type=1, code=103, value=0
[AT Translated Set 2 keyboard] type=0, code=0, value=0
[AT Translated Set 2 keyboard] type=4, code=4, value=205
[AT Translated Set 2 keyboard] type=1, code=106, value=1
[AT Translated Set 2 keyboard] type=0, code=0, value=0
^[[C[AT Translated Set 2 keyboard] type=4, code=4, value=205
[AT Translated Set 2 keyboard] type=1, code=106, value=0
[AT Translated Set 2 keyboard] type=0, code=0, value=0
```

1. `read_input` — читает события с одного устройства, выводит поля, получает имя и физ. путь через `ioctl`.
2. Обработка ошибок — есть проверка `open`, `ioctl`, `read`.
3. `poll_inputs` — отслеживает несколько устройств, использует `poll()`, определяет источник события, выводит имя устройства.

Контрольные вопросы

1. В чём принципиальное отличие символьных устройств от блочных?
Приведите по два примера каждого типа.

Принципиальное отличие:

- Символьные устройства (character devices) передают данные побайтово (или по символам), без буферизации на уровне ядра, и не поддерживают произвольный доступ (seek). Обычно используются для потоковых устройств.
- Блочные устройства (block devices) передают данные блоками фиксированного размера (например, 512 Б или 4 КБ), поддерживают произвольный доступ (lseek) и используют кэширование на уровне ядра.

Примеры:

- Символьные: `/dev/ttyS0` (последовательный порт), `/dev/input/event0` (устройство ввода), `/dev/urandom`.
- Блочные: `/dev/sda` (жёсткий диск), `/dev/loop0` (loop-устройство).

2. Почему для ожидания событий от нескольких источников данных `poll()` является предпочтительнее, чем `busy-wait` цикл? Опишите сценарий, где это критически важно.

Причина:

`poll()` блокирует процесс до появления данных, освобождая CPU для других задач. В отличие от этого, `busy-wait` (цикл с постоянной проверкой) непрерывно расходует процессорное время, даже когда данных нет.

Критический сценарий:

Встраиваемая система (например, IoT-устройство на ARM с ограничением энергопотребления). Использование `busy-wait` приведёт к:

- Резкому росту потребления энергии,
- Перегреву,
- Сокращению срока службы батареи.

Использование `poll()` позволяет ядру перевести CPU в спящий режим, пока нет событий, что критично для энергоэффективности.

3. Может ли вызов `read()` для символьного устройства заблокировать процесс? Если да, то как этого избежать?

Да, вызов `read()` для символьного устройства может заблокировать процесс, если:

Внутренний буфер устройства пуст, и драйвер ожидает поступления данных (например, ожидание нажатия клавиши на `/dev/input/eventX`).

Как избежать блокировки:

- Открыть файл с флагом `O_NONBLOCK`:

```
int fd = open("/dev/input/event0", O_RDONLY | O_NONBLOCK);
```

- Тогда `read()` вернёт -1 и установит `errno = EAGAIN` (или `EWOULDBLOCK`), если данных нет.
- Альтернатива: использовать `poll()/select()` для ожидания данных до вызова `read()`.

4. Объясните, почему для работы с `ioctl` и структурами из `/dev/input/` требуются специфичные для Linux заголовочные файлы. Является ли такой код переносимым на другие POSIX-системы (например, macOS или FreeBSD)?

Причина:

Интерфейс устройств ввода (`/dev/input/eventX`), структура `struct input_event`, константы (`EV_KEY`, `EVIOCGNAME`) и `ioctl`-команды (`EVIOCGNAME`, `EVIOCGPHYS`) — это специфика Linux Input Subsystem, реализованная в ядре Linux.

Переносимость:

- Такой код не переносим на другие POSIX-системы:
- FreeBSD использует `devd` и `ukbd/ums`, события читаются через другие механизмы (например, `libinput` или `evdev` — только если включён совместимый драйвер).
- macOS (на базе Darwin/XNU) не имеет `/dev/input/eventX`. Устройства ввода доступны только через IOKit (C++ фреймворк в режиме ядра) или HID Manager API (userspace).
- Даже реализация `ioctl` и его команд не стандартизирована POSIX — это расширение, зависящее от ОС.